

Нов Български Университет

BSNINF01 Информатика / Компютърно програмиране

CSCB766 Програмиране с ASP .NET

ASP.NET Проект

ChatAppClone - чат платформа изградена с .NET Core 8.0, EF Core and SignalR

Изготвил: Георги Костадинов

Факултетен номер: F108668

Специалност: BSNINF01 Информатика

Курс: CSCB766 Програмиране с ASP .NET

Преподавател: доц. д-р Лъчезар Томов

2026 г.

Съдържание

Описание на проекта.....	2
Ръководство на потребителя	4
Описание на базата данни	8
Описание на външни компоненти	11
Заклучение.....	14

Описание на проекта

ChatAppClone е уеб приложение за комуникация в реално време, разработено с ASP.NET Core MVC, Entity Framework Core, Microsoft Identity и SignalR. Основната цел на проекта е да предостави удобна платформа, в която регистрирани потребители могат да откриват други потребители, да започват директни чатове, да изпращат съобщения в реално време и да управляват своя профил.

Проектът е реализиран като MVC приложение с отделени слоеве за контролери, бизнес логика, достъп до данни, модели, изгледи и помощни услуги. Това улеснява поддръжката на кода и разделя ясно отговорностите между различните части на системата. Потребителският интерфейс е изграден с Razor views, HTML, CSS, Bootstrap и JavaScript, а реалновременната комуникация се осъществява чрез SignalR hubs.

Основните функционалности на приложението са:

- Регистрация на потребител с валиден имейл адрес.
- Потвърждение на потребителски акаунт чрез имейл.
- Вход и изход от системата.
- Възстановяване и смяна на парола.
- Управление на потребителски профил.
- Качване и изтриване на профилна снимка.
- Разглеждане и търсене на други потребители.
- Стартиране на директен чат с избран потребител.
- Изпращане и получаване на съобщения в реално време.
- Преглед на всички активни чатове на потребителя.
- Изтриване на чат.
- Получаване на известия при създаване или изтриване на чат.
- Обработка на грешки чрез custom middleware и отделни error страници.

Приложението използва Microsoft Identity за регистрация, вход, управление на акаунт и роли. Потребителят може да управлява стандартни настройки като имейл, потребителско име, парола и лични данни. Допълнително е добавена възможност за профилна снимка, която се съхранява във външна cloud услуга.

Чат функционалността е основната част на проекта. След вход в системата потребителят може да отвори страницата Explore, където се показват други регистрирани потребители. Оттам може да бъде стартиран директен чат. При създаване на чат се създава запис в базата данни, добавят се участниците и другият потребител получава известие в реално време. Ако чатът вече съществува, приложението не създава дублиращ разговор.

След като потребителят отвори страницата Chats, той вижда списък с разговорите си. При избор на конкретен чат се зареждат съобщенията, участниците и детайлите за разговора. Съобщенията се изпращат чрез API заявка, записват се в базата данни и след това се разпространяват към всички участници в съответната SignalR група. Така получателят вижда новото съобщение веднага, без да презарежда страницата.

В проекта има и система за известия. Тя се използва при важни действия, например когато потребител бъде добавен към нов чат или когато съществуващ чат бъде изтрит. Известията се записват в базата данни и могат да бъдат показвани на потребителя. За моменталното им получаване се използва отделен SignalR hub.

Приложението е подходящ пример за цялостен ASP.NET Core проект, защото комбинира потребителска автентикация, база данни, реалновременна комуникация, външни услуги, клиентски JavaScript и backend логика.

Ръководство на потребителя

Тази част описва основните действия, които един потребител може да извърши в приложението. Поради простия и интуитивен характер на чат приложението не са включени screenshots. Екраните следват стандартна навигация: начална страница, регистрация/вход, профил, Explore страница и Chats страница.

Достъп до приложението

При отваряне на приложението потребителят попада на началната страница. От навигационното меню може да избере вход, регистрация, информация за приложението или други публични страници като About и Privacy.

Нерегистриран потребител има ограничен достъп. За да използва чат функционалността, той трябва да създаде акаунт и да влезе в системата.

Регистрация

За създаване на нов акаунт потребителят избира страницата за регистрация. Там въвежда необходимите данни, например имейл, потребителско име и парола. След изпращане на формата приложението създава нов Identity потребител.

Проектът изисква потвърден акаунт, затова след регистрация потребителят трябва да потвърди своя имейл адрес. Потвърждението се изпраща чрез email service. Това предпазва системата от невалидни регистрации и гарантира, че потребителят има достъп до посочения имейл.

Вход в системата

След потвърждение на акаунта потребителят може да влезе в системата чрез страницата Login. Необходимо е да въведе своите данни за достъп. При успешно влизане приложението го разпознава като автентикиран потребител и му дава достъп до основните функционалности.

При вход може да бъде използвана опцията Remember me, ако потребителят желае сесията да бъде запазена за по-дълго време.

Възстановяване на парола

Ако потребителят забрави паролата си, може да използва Forgot Password страницата. След въвеждане на имейл системата изпраща инструкции за възстановяване. Чрез получения линк потребителят може да зададе нова парола.

Тази функционалност също използва email service и стандартните механизми на Microsoft Identity.

Управление на профил

След вход потребителят може да отвори страницата за управление на профила. Там може да промени потребителски данни, имейл и парола. Приложението поддържа и управление на профилна снимка.

За качване на профилна снимка потребителят избира файл от устройството си. Поддържат се изображения във формати `.jpg`, `.jpeg` и `.png`. След потвърждение файлът се качва в Cloudinary, а в базата данни се записва URL адресът на снимката и нейният public id. Ако потребителят изтрие снимката, тя се премахва от Cloudinary и профилът се връща към стандартното изображение.

Търсене на потребители

Страницата Explore показва списък с други регистрирани потребители. Текущият потребител не вижда себе си в списъка. Налична е възможност за търсене по потребителско име или имейл.

Списъкът е странициран, което означава, че при повече потребители те се показват на части. Това подобрява четимостта и намалява количеството данни, което се зарежда наведнъж.

От всяка потребителска карта може да бъде стартиран директен чат. Ако между двамата потребители вече има съществуващ чат, системата не създава нов, а пренасочва към страницата с чатове.

Стартиране на чат

Когато потребителят избере друг потребител от Explore страницата и стартира чат, приложението създава нов разговор. В базата данни се създава запис за чата и отделни записи, които свързват двамата участници с него.

Другият потребител получава известие, че е добавен към чат. Ако е онлайн, известието се получава в реално време чрез SignalR. Новият чат се появява и в списъка с разговори.

Използване на страницата Chats

Страницата Chats е основният работен екран на приложението. Тя съдържа списък с разговорите на потребителя. Всеки чат се показва с име, изображение, последно съобщение и информация за последна активност.

При избор на чат се зарежда детайлната част на разговора. Там се показват участниците и историята на съобщенията. Зареждането на конкретен чат става без пълно презареждане на страницата, като се използва частичен изглед.

Изпращане на съобщения

За да изпрати съобщение, потребителят избира чат, въвежда текст в полето за съобщение и натиска бутона за изпращане. Поддържа се и изпращане чрез клавиша Enter.

След изпращане съобщението се записва в таблицата Messages. След това backend частта изпраща съобщението към всички участници в съответната SignalR група. Така съобщението се появява веднага в прозореца на разговора както при изпращача, така и при получателя.

Интерфейсът обновява и последното съобщение в списъка с чатове, както и времето на последната активност.

Изтриване на чат

Потребителят може да изтрие чат, който вече не желае да вижда. При изтриване системата премахва чата от базата данни. Понеже връзките към участниците и съобщенията са свързани с чата, те също се премахват според конфигурацията на базата.

След изтриване всички участници получават известие. Ако са онлайн, интерфейсът им се обновява чрез SignalR и изтритият чат се премахва от списъка.

Известия

Известията служат за информиране на потребителя за важни действия. Например потребителят може да получи известие, когато друг потребител го добави към чат или когато чат бъде изтрит.

Известията се записват в базата данни, така че не се губят веднага след създаване. За моментално показване на ново известие се използва NotificationHub.

Грешки и защита

Ако потребителят се опита да достъпи страница без необходимите права или възникне грешка, приложението показва подходяща error страница. Има отделни изгледи за често срещани HTTP статуси като 400, 401, 403, 404 и 500.

Неавтентикирани потребители не могат да използват чат функционалностите. При липса на валиден потребителски идентификатор системата пренасочва към подходяща страница за грешка.

Описание на базата данни

Проектът използва SQL Server база данни чрез Entity Framework Core. Основният контекст е `'ChatAppCloneDbContext'`, който наследява `'IdentityDbContext<ApplicationUser, ApplicationRole, string>'`. Това означава, че базата включва стандартните таблици на Microsoft Identity, както и допълнителни таблици, създадени специално за чат приложението.

При стартиране на приложението се изпълняват миграциите към базата данни. След това се стартира seeding логика, която може да добави начални данни, ако базата е празна.

Identity таблици

Microsoft Identity създава таблици за потребители, роли, claims, logins и tokens. Най-важната таблица е `'AspNetUsers'`, която съхранява данните за регистрираните потребители.

В проекта стандартният Identity потребител е разширен чрез класа `'ApplicationUser'`. Допълнителните полета са:

- `'ProfilePictureUrl'` - URL адрес към профилната снимка на потребителя.
- `'ProfilePicturePublicId'` - идентификатор на снимката в Cloudinary, нужен при замяна или изтриване.
- `'CreatedOn'` - дата на създаване на потребителя.
- `'ModifiedOn'` - дата на последна промяна.

`'ApplicationUser'` има връзки към съобщения, чатове и известия.

Таблица Chats

Таблицата `'Chats'` съхранява основната информация за всеки разговор. Основните полета са:

- `'Id'` - уникален идентификатор на чата от тип Guid.
- `'Name'` - име на чата. При директен чат се формира от имената на участниците.
- `'IsGroupChat'` - указва дали чатът е групов. В текущата реализация основният сценарий е директен чат между двама потребители.
- `'ImageUrl'` - изображение на чата.
- `'LastMessage'` - последното съобщение, показвано в списъка с разговори.

- 'LastActive' - време на последна активност в чата.
- 'CreatedOn' и 'ModifiedOn' - общи полета за проследяване на създаване и промяна.

Един чат може да има много съобщения и много участници.

Таблица Messages

Таблицата 'Messages' съхранява всички изпратени съобщения. Основните полета са:

- 'Id' - уникален идентификатор на съобщението.
- 'Content' - текстът на съобщението.
- 'IsSeen' - дали съобщението е видно.
- 'CreatorId' - идентификатор на потребителя, който е изпратил съобщението.
- 'ChatId' - идентификатор на чата, към който принадлежи съобщението.
- 'CreatedOn' и 'ModifiedOn' - данни за създаване и промяна.

Връзките са следните: един потребител може да има много съобщения, а един чат може да съдържа много съобщения. При зареждане на чат съобщенията се подреждат по дата на създаване.

Таблица UsersChats

Таблицата 'UsersChats' е междинна таблица за връзката много към много между потребители и чатове. Тя показва кои потребители участват в даден чат.

Основните полета са:

- 'Id' - уникален идентификатор на записа.
- 'UserId' - идентификатор на потребителя.
- 'ChatId' - идентификатор на чата.
- 'CreatedOn' и 'ModifiedOn' - общи полета за проследяване.

Чрез тази таблица системата може да намери всички чатове на конкретен потребител, както и всички участници в конкретен чат.

Таблица Notifications

Таблицата 'Notifications' съхранява известията за потребителите. Основните полета са:

- 'Id' - уникален идентификатор на известието.
- 'UserId' - потребителят, за когото е предназначено известието.
- 'Content' - текст на известието.
- 'Url' - възможен адрес, към който известието може да насочи потребителя.
- 'IsRead' - дали известието е прочетено.
- 'CreatedOn' и 'ModifiedOn' - данни за създаване и промяна.

Един потребител може да има много известия. Известията могат да се показват страницирано.

Основни връзки в базата

Основните връзки между таблиците са:

- Един 'ApplicationUser' има много 'Message' записи.
- Един 'Chat' има много 'Message' записи.
- Един 'ApplicationUser' участва в много 'Chat' записи чрез 'UserChat'.
- Един 'Chat' има много участници чрез 'UserChat'.
- Един 'ApplicationUser' има много 'Notification' записи.

При изтриване на чат свързаните записи в 'UsersChats' се премахват каскадно. Това пази базата от останали невалидни връзки към несъществуващи чатове.

Описание на външни компоненти

Проектът използва няколко външни компонента и библиотеки, които осигуряват основни технически възможности.

ASP.NET Core MVC и Razor Views

ASP.NET Core MVC е основната рамка, върху която е разработено приложението. Контролерите обработват заявките, services съдържат бизнес логиката, а Razor views изграждат HTML интерфейса.

Използвани са и Razor Pages в Identity area, тъй като Microsoft Identity предоставя готови страници за регистрация, вход, управление на акаунт, смяна на парола и потвърждение на имейл.

Microsoft Identity

Microsoft Identity се използва за автентикация и управление на потребителски акаунти. Чрез него са реализирани регистрация, вход, изход, роли, потвърждение на имейл, забравена парола и управление на профила.

Identity е интегриран с Entity Framework Core, затова потребителите и свързаните с тях данни се съхраняват в SQL Server.

Entity Framework Core и SQL Server

Entity Framework Core се използва като ORM за работа с базата данни. Чрез него C# класовете се преобразуват в таблици, а заявките към базата се пишат чрез LINQ.

SQL Server е използваната релационна база данни. Проектът съдържа миграции, чрез които структурата на базата може да бъде създадена и обновявана автоматично.

SignalR

SignalR е ключов компонент за реалновременната комуникация. В проекта има два hub-а:

- 'ChatHub' - използва се за присъединяване към чат групи, напускане на групи и получаване на съобщения в реално време.
- 'NotificationHub' - използва се за изпращане на известия към конкретни потребители.

Когато потребител изпрати съобщение, backend частта го записва в базата и след това чрез SignalR го изпраща към всички участници в съответния чат. Това премахва нуждата от ръчно презареждане на страницата.

Cloudinary

Cloudinary се използва за съхранение на профилни снимки. Вместо изображенията да се пазят директно на сървъра, те се качват в cloud услуга. В базата данни се пазят само URL адресът на снимката и public id.

Това улеснява управлението на файловете, намалява натоварването върху приложението и позволява по-лесна подмяна или изтриване на профилни снимки.

MailKit и MimeKit

MailKit и MimeKit се използват за изпращане на имейли. В проекта тази функционалност е важна за потвърждение на акаунт и възстановяване на парола.

Email service изгражда HTML съобщение, свързва се със SMTP сървър и изпраща имейла към потребителя.

Serilog

Serilog се използва за логване на действия и грешки. Приложението конфигурира Serilog като host-level logger и записва логове във файлове. Това е полезно при проследяване на проблеми и при наблюдение на важни действия като създаване на съобщение, зареждане на чат или изтриване на чат.

Bootstrap, CSS и JavaScript

За потребителския интерфейс са използвани HTML, CSS, Bootstrap и JavaScript. Bootstrap помага за изграждане на responsive layout и стандартни UI компоненти. Собствените CSS файлове оформят страниците за началния екран, чат, профил, login/register, известия и error страници.

JavaScript се използва най-вече в чат страницата. Той създава SignalR връзка, зарежда избрания чат, изпраща API заявки, добавя нови съобщения в интерфейса и обновява списъка с разговори.

LibMan и SignalR client библиотека

Проектът използва LibMan за добавяне на клиентската SignalR библиотека в ``wwwroot/lib/microsoft/signalr``. Тази библиотека позволява JavaScript кодът в брауъра да се свързва със SignalR hubs на сървъра.

NUnit, Moq и InMemory Database

В проекта са добавени библиотеки за unit testing: NUnit, Moq и Entity Framework Core InMemory. Те позволяват тестване на business logic без реална SQL Server база. InMemory базата е полезна за тестови сценарии, а Moq позволява заместване на зависимости при тестове.

Заклучение

ChatAppClone е цялостно чат приложение, което съчетава стандартни уеб функционалности и реалновременна комуникация. Проектът включва регистрация и вход, управление на профил, търсене на потребители, директни чатове, изпращане на съобщения, известия, работа с база данни и интеграция с външни услуги.

Най-важната част на приложението е комуникацията в реално време чрез SignalR. Благодарение на нея потребителите получават нови съобщения и известия веднага, без да презареждат страницата. Базата данни е организирана ясно около потребители, чатове, съобщения, участници и известия, което прави проекта лесен за разбиране и бъдещо разширяване.